

Pruebas de:

Gabriel Bustos:

Herramienta utilizada: Se utilizó el framework pytest, apoyado por las librerías pytest-mock (para aislar la base de datos) y pytest-cov (para medir el coverage).

Funcionalidad validada: Se probó el Servicio Central de Autenticación (AuthService)., ya que gestiona el ingreso al sistema, la validación de contraseñas cifradas y el ciclo de vida de los JSON Web Tokens (JWT).

Casos Límites (Edge Cases) cubiertos y documentados:

Caso Límite 1: Intento de login con un correo válido pero con un usuario inhabilitado (activo = False).

Caso Límite 2: Intento de login con una contraseña errónea para un usuario existente.

Caso Límite 3: Intento de validación con un token criptográfico expirado (ExpiredSignatureError).

Caso Límite 4: Intento de validación con un token malformado o inválido (InvalidTokenError).

Resultados:

```
PS C:\Users\PC\project-backend\Mentora> python -m pytest --ds=config.settings usuarios/tests/test_services.py --cov=usuarios.services --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
django: version: 6.0.4, settings: config.settings (from option)
rootdir: C:\Users\PC\project-backend\Mentora
plugins: cov-7.1.0, django-4.12.0, mock-3.15.1
collected 7 items

usuarios\tests\test_services.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.7-final-0

Name                Stmts  Miss  Cover   Missing
-----
usuarios\services.py   35     4    89%   10-11, 34, 38
TOTAL                  35     4    89%

===== 7 passed in 0.15s =====
PS C:\Users\PC\project-backend\Mentora>
```

Alexander Mosquera Ramirez:

Herramienta utilizada: Se utilizó el framework **pytest** en conjunto con la librería de emulación types.ModuleType para inyectar un entorno de aislamiento total sobre el sistema de archivos (sys.modules). Esta estrategia permite verificar de manera pura y matemática la lógica de negocio de los servicios en la memoria interna del

intérprete, cumpliendo estrictamente con la directriz de no inicializar ni realizar lecturas/escrituras en el motor de base de datos PostgreSQL de producción.

Funcionalidades validadas: Se probó el núcleo lógico del agendamiento dentro de TutoringService y las reglas financieras/operativas de penalizaciones en PenalizacionService. El alcance cubre el control de ventanas de tiempo de reserva, duraciones máximas de sesión y algoritmos de colisión de franjas horarias.

Casos límites (Edge Cases) cubiertos y documentados:

- Caso Límite 1: Intento de agendamiento de una tutoría en una fecha u hora cronológicamente pasada en comparación con la estampa de tiempo actual del servidor (ValidationError).
- Caso Límite 2: Intento de registro de una sesión que excede el límite máximo operativo permitido por la plataforma (sesiones continuas con una duración mayor a 4 horas).
- Caso Límite 3: Cancelación tardía o extrema de una cita académica a falta de solo 30 minutos de su inicio, forzando la activación del estado de penalización estricto (multa del 100%).
- Caso Límite 4: Solapamiento o intersección parcial de tiempo (colisión de 15 minutos) entre un bloque de horario ya reservado y una nueva propuesta enviada por un tutor.

```
PS C:\Users\holaa\Documents\Mentora> uv run --with pytest --with django --with djangorestframework python -m pytest Mentora/test_alex.py
Installed 11 packages in 4.96s
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\holaa\Documents\Mentora
collected 4 items

Mentora\test_alex.py .... [100%]
===== 4 passed in 1.30s =====
PS C:\Users\holaa\Documents\Mentora>
```

Diego Jiménez:

Notas:

Herramienta utilizada: Se utilizó la herramienta Pytest, compatible con Python y ampliamente utilizada para la automatización de pruebas en proyectos Django.

Funcionalidad validada: Se validaron las reglas de negocio asociadas al registro de tutorías (SessionRecord), específicamente las restricciones sobre calificaciones permitidas (límites mínimos y máximos), manejo de observaciones y restricciones de longitud de texto.

Casos límites (Edge Cases) cubiertos y documentados:

Caso Límite 1: Calificación inferior al mínimo permitido (0.5). Resultado esperado: ValidationError.

Caso Límite 2: Calificación superior al máximo permitido (5.5). Resultado esperado: ValidationError.

Caso Límite 3: Calificación negativa (-2.5). Resultado esperado: ValidationError.

Caso Límite 4: Calificación extremadamente grande (999999). Resultado esperado: ValidationError.

Caso Límite 5: Observación superior a 1000 caracteres. Resultado esperado: ValidationError.

Caso Límite 6: Observación exactamente de 1000 caracteres. Resultado esperado: Se acepta correctamente.

Caso Límite 7: Calificación exactamente igual al límite mínimo (1.0). Resultado esperado: Se acepta correctamente.

Caso Límite 8: Calificación exactamente igual al límite máximo (5.0). Resultado esperado: Se acepta correctamente.

```
PS C:\Users\diego\Documents\mentora-recuperacion\mentora> python -m coverage run -m pytest tutorias/tests
/test_session_record_serializer.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.0, pluggy-1.6.0
django: version: 6.0.4, settings: config.settings (from ini)
rootdir: C:\Users\diego\Documents\mentora-recuperacion\mentora
configfile: pytest.ini
plugins: django-4.12.0, mock-3.15.1
collected 10 items

tutorias\tests\test_session_record_serializer.py ..... [100%]

===== 10 passed in 0.17s =====
```

Nota: Son 10 test, pero 8 verifican casos límites.

Notificaciones:

Herramienta utilizada: Se utilizó Pytest con mocks para evitar dependencias externas y acceso a base de datos.

Funcionalidad validada: Se verificó el comportamiento del servicio encargado de generar recordatorios automáticos para tutorías programadas (Notificaciones).

Casos límites (Edge Cases) cubiertos y documentados:

Caso Límite 1: No existen tutorías programadas. Resultado esperado: Se generan 0 notificaciones.

Caso Límite 2: Existe una tutoría con un único participante. Resultado esperado: Se genera 1 notificación.

Caso Límite 3: Existe una tutoría con múltiples participantes. Resultado esperado: Se genera una notificación por participante.

Caso Límite 4: Existen múltiples tutorías programadas. Resultado esperado: Se generan todas las notificaciones correspondientes.

Caso Límite 5: Tutoría sin participantes registrados. Resultado esperado: No se generan notificaciones.

Caso Límite 6: El mensaje contiene el nombre de la materia. Resultado esperado: El texto incluye la asignatura.

Caso Límite 7: El mensaje contiene la hora de inicio. Resultado esperado: El texto incluye la hora programada.

Caso Límite 8: La notificación utiliza el usuario asociado al participante. Resultado esperado: El usuario enviado coincide con el participante.

Caso Límite 9: El mensaje inicia con la palabra "Recordatorio". Resultado esperado: Se mantiene el formato establecido.

Caso Límite 10: Se invoca una llamada a creación por cada participante. Resultado esperado: Número de llamadas igual al número de participantes.

```
PS C:\Users\diego\Documents\mentora-recuperacion\mentora> python -m coverage run -m pytest tutorias/tests
/test_notification_service.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.0, pluggy-1.6.0
django: version: 6.0.4, settings: config.settings (from ini)
rootdir: C:\Users\diego\Documents\mentora-recuperacion\mentora
configfile: pytest.ini
plugins: django-4.12.0, mock-3.15.1
collected 10 items

tutorias\tests\test_notification_service.py ..... [100%]

===== 10 passed in 0.16s =====
```

Asistencia (Ausencias):

Herramienta utilizada: Se utilizó Pytest para validación de estados.

Funcionalidad validada: Se verificó la validez de los estados de asistencia (Attendance) permitidos por el sistema.

Casos límites (Edge Cases) cubiertos y documentados:

Caso Límite 1: Estado PRESENTE. Resultado esperado: Válido.

- Caso Límite 2: Estado AUSENTE. Resultado esperado: Válido.
- Caso Límite 3: Estado TARDE. Resultado esperado: Válido.
- Caso Límite 4: Cadena vacía. Resultado esperado: ValidationError.
- Caso Límite 5: Valor aleatorio (XYZ). Resultado esperado: ValidationError.
- Caso Límite 6: Valor numérico. Resultado esperado: ValidationError.
- Caso Límite 7: Texto en minúsculas. Resultado esperado: ValidationError.
- Caso Límite 8: Cadena con espacios. Resultado esperado: ValidationError.
- Caso Límite 9: Valor nulo. Resultado esperado: ValidationError.
- Caso Límite 10: Texto excesivamente largo. Resultado esperado: ValidationError.

```
PS C:\Users\diego\Documents\mentora-recuperacion\mentora> python -m coverage run -m pytest tutorias/tests/test_attendance_serializer.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.0, pluggy-1.6.0
django: version: 6.0.4, settings: config.settings (from ini)
rootdir: C:\Users\diego\Documents\mentora-recuperacion\mentora
configfile: pytest.ini
plugins: django-4.12.0, mock-3.15.1
collected 10 items

tutorias\tests\test_attendance_serializer.py ..... [100%]

===== 10 passed in 0.17s =====
```

Pablo Rosales

Herramienta	Uso
Pytest	Framework principal para ejecutar las pruebas unitarias.
unittest.mock	Simular objetos y dependencias mediante MagicMock y patch .
Django REST Framework (APIRequestFactory)	Crear peticiones HTTP simuladas para probar las vistas sin necesidad de levantar el servidor.
pytest-django	Integrar Pytest con Django y cargar la configuración del proyecto.

Funcionalidades Evaluadas

Las pruebas unitarias fueron desarrolladas sobre la clase `TutoringSessionViewSet`, correspondiente al módulo de gestión de tutorías. Se seleccionaron funcionalidades consideradas críticas para el correcto funcionamiento del sistema.

Caso de Prueba 1: Cancelación de una Tutoría Ya Cancelada

Nombre de la prueba:

`test_cancel_already_cancelled_session_returns_error`

Objetivo:

Verificar que el sistema impida la cancelación de una tutoría cuyo estado ya sea "cancelada".

Resultado esperado:

El sistema debe responder con un código HTTP 400 (Bad Request) y mostrar el mensaje correspondiente indicando que la tutoría ya fue cancelada.

Resultado obtenido:

Prueba aprobada exitosamente.

Caso de Prueba 2: Cancelación de una Tutoría sin Motivo

Nombre de la prueba:

`test_cancel_session_without_reason_returns_error`

Objetivo:

Validar que el sistema exija el ingreso de un motivo de cancelación antes de procesar la solicitud.

Resultado esperado:

El sistema debe responder con un código HTTP 400 (Bad Request) indicando que es obligatorio proporcionar un motivo de cancelación.

Resultado obtenido:

Prueba aprobada exitosamente.

Caso de Prueba 3: Reprogramación Exitosa de una Tutoría

Nombre de la prueba:

`test_reschedule_session_successfully`

Objetivo:

Verificar que una tutoría pueda ser reprogramada correctamente cuando se proporcionan los datos requeridos para la nueva fecha y horario.

Resultado esperado:

El sistema debe crear una nueva tutoría con los datos suministrados, actualizar el estado de la tutoría original a "reprogramada" y retornar un código HTTP 200 (OK).

Resultado obtenido:

Prueba aprobada exitosamente.

Casos Límite Evaluados

Durante la ejecución de las pruebas se verificaron los siguientes escenarios límite:

- Intento de cancelar una tutoría que ya había sido cancelada previamente.
- Intento de cancelar una tutoría sin especificar un motivo de cancelación.
- Reprogramación de una tutoría válida verificando la actualización correcta de su estado.

Evidencia de Ejecución

Las pruebas fueron ejecutadas mediante el siguiente comando:

None

```
pytest tutorias/tests/test_tutoring_session.py -v
```

Resultado obtenido:

```

(env) PS D:\project-backend\Mentorax> pytest tutorias/tests/test_tutoring_session.py -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.0, pluggy-1.6.0 -- D:\project-backend\env\Scripts\python.exe
cachedir: .pytest_cache
django: version: 4.2.11, settings: config.settings (from ini)
rootdir: D:\project-backend\Mentorax
configfile: pytest.ini
plugins: django-4.12.0
collected 3 items

tutorias/tests/test_tutoring_session.py::test_cancel_already_cancelled_session_returns_error PASSED [ 33%]
tutorias/tests/test_tutoring_session.py::test_cancel_session_without_reason_returns_error FAILED [ 66%]
tutorias/tests/test_tutoring_session.py::test_reschedule_session_successfully FAILED [100%]

===== FAILURES =====
_____ test_cancel_session_without_reason_returns_error _____

```

```

collected 3 items

tutorias/tests/test_tutoring_session.py::test_cancel_already_cancelled_session_returns_error PASSED [ 33%]
tutorias/tests/test_tutoring_session.py::test_cancel_session_without_reason_returns_error PASSED [ 66%]
tutorias/tests/test_tutoring_session.py::test_reschedule_session_successfully PASSED [100%]

```

```

tutorias\tests\test_tutoring_session.py ... [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.14.3-final-0 _____

Name                                Stmts  Miss  Cover
-----
tutorias\admin.py                    0      0  100%
tutorias\apps.py                     3      0  100%
tutorias\migrations\0001_initial.py  6      6   0%
tutorias\migrations\__init__.py      0      0  100%
tutorias\models.py                  74      0  100%
tutorias\serializers.py             34      0  100%
tutorias\tests.py                   16     16   0%
tutorias\tests\test_tutoring_session.py 45      0  100%
tutorias\urls.py                    13     13   0%
tutorias\views.py                   89     19   79%
-----
TOTAL                                280     54   81%

```